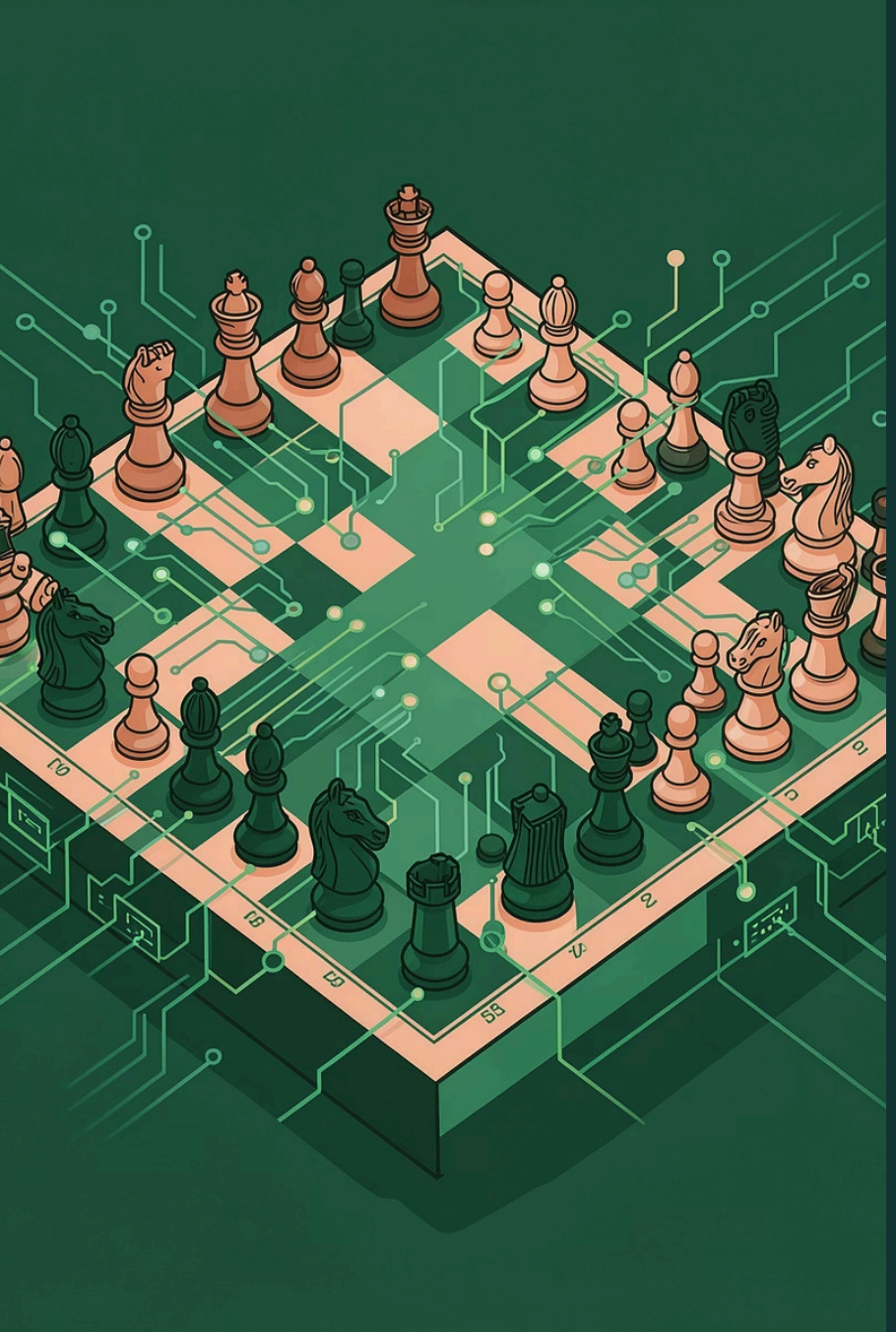


CUDA Chess Vision

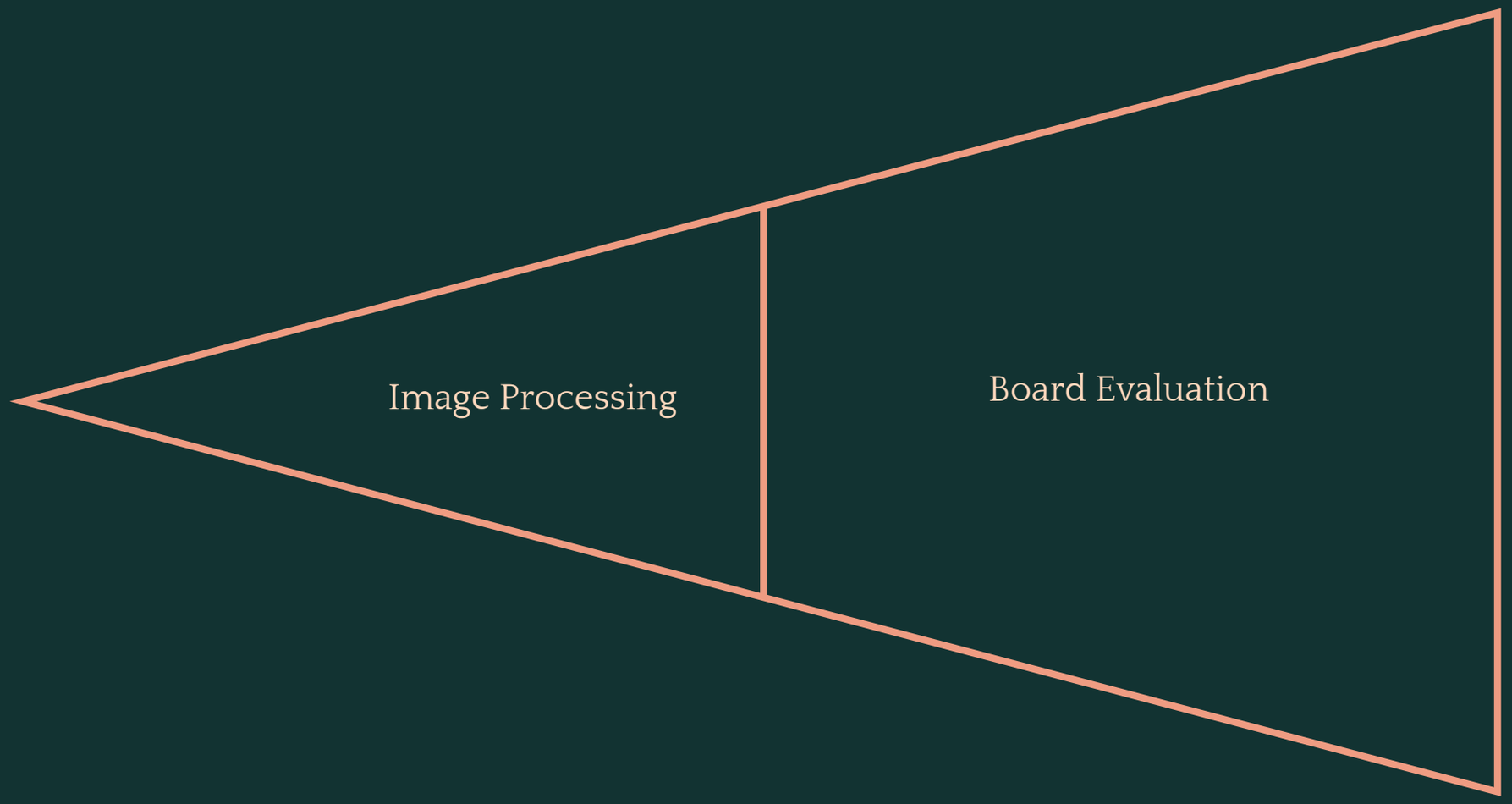
A GPU-accelerated chess board image processing and position evaluation pipeline — leveraging **NPP**, **cuBLAS**, **cuFFT**, and **Thrust** in a unified two-stage pipeline.

GPU PIPELINE | CUDA TOOLKIT 11.X / 12.X | NVIDIA T4 · SM_75

Made with GAMMA

Two-Stage GPU Pipeline

Each chess board PNG flows through two tightly coupled GPU stages — from raw pixels to ranked position scores.



Stage 1 produces a 64-value intensity map per board; Stage 2 evaluates all boards in batch and outputs ranked scores.

Made with GAMMA

Stage 1 — Image Processing

NPP Primitives

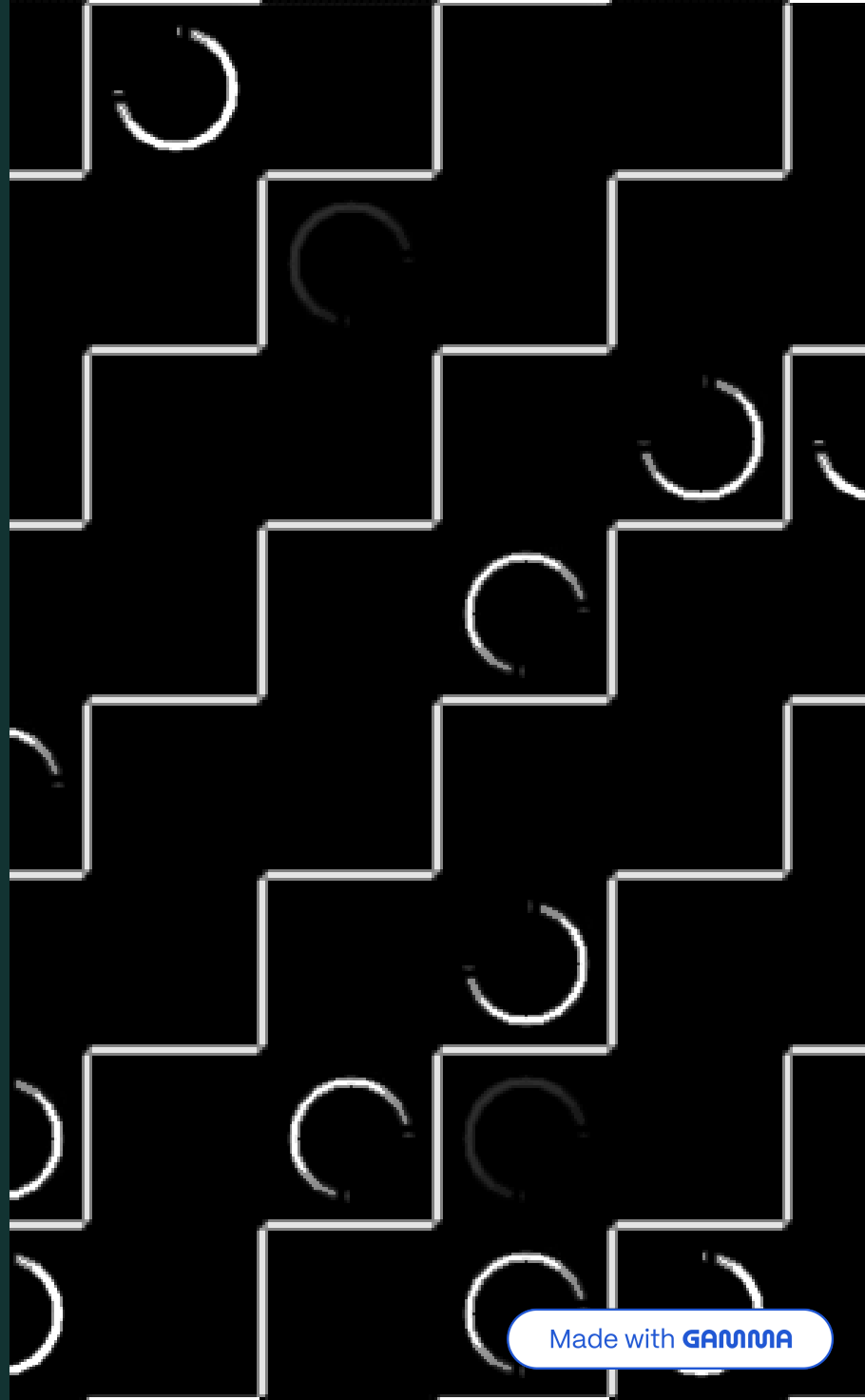
- RGB → grayscale
- 3×3 Gaussian blur
- Horizontal & vertical Sobel

Custom CUDA Kernels

- Gradient magnitude $\sqrt{(Gx^2+Gy^2)}$
- 256-bin shared-mem histogram
- Histogram equalization LUT
- 8×8 per-square intensity reduction

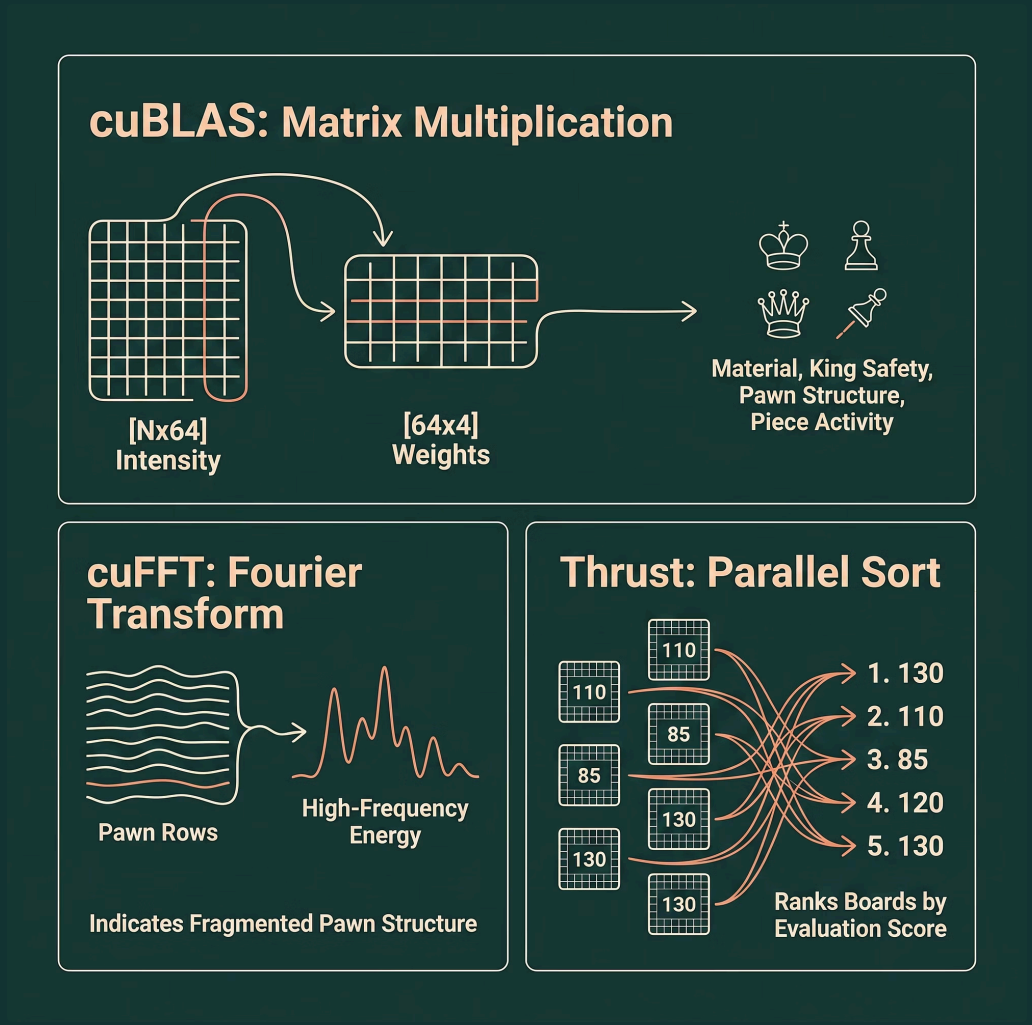
Output

- Processed grayscale PNG
- 64-value intensity map (float per square)
- ~28 ms per board on T4



Made with GAMMA

Stage 2 — Board Evaluation



Batch Processing

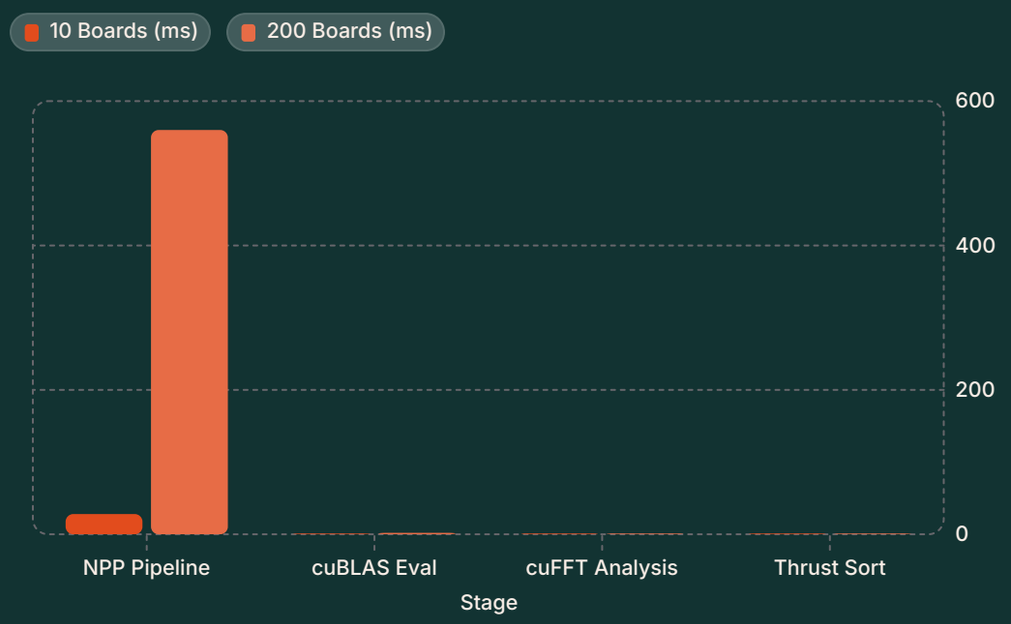
All N intensity maps are evaluated together — no per-board serialization.

- **cuBLAS** — `cublasSgemm` computes [N×4] feature scores
- **cuFFT** — `cufftPlanMany` runs N independent 8-point FFTs in one call
- **Thrust** — `sort_by_key` ranks boards by aggregated score

> ⓘ Total Stage 2 time for 200 boards: **<4 ms** on NVIDIA T4.

Made with GAMMA

Performance & Lessons Learned



Key Engineering Takeaways

NPP API Portability

Replaced deprecated `nppiEqualizeHist` with portable custom histogram → LUT → apply kernels.

cuBLAS Column-Major

Transpose both operands mentally for row-major [N×64]×[64×4] via `cublasSgemm`.

cuFFT Batch Mode

`cufftPlanMany` handles N independent FFTs in one call — far faster than N separate plans.

Thrust + Raw Pointers

Wrap `cudaMalloc` pointers via iterator construction to avoid double-allocation.

Made with GAMMA